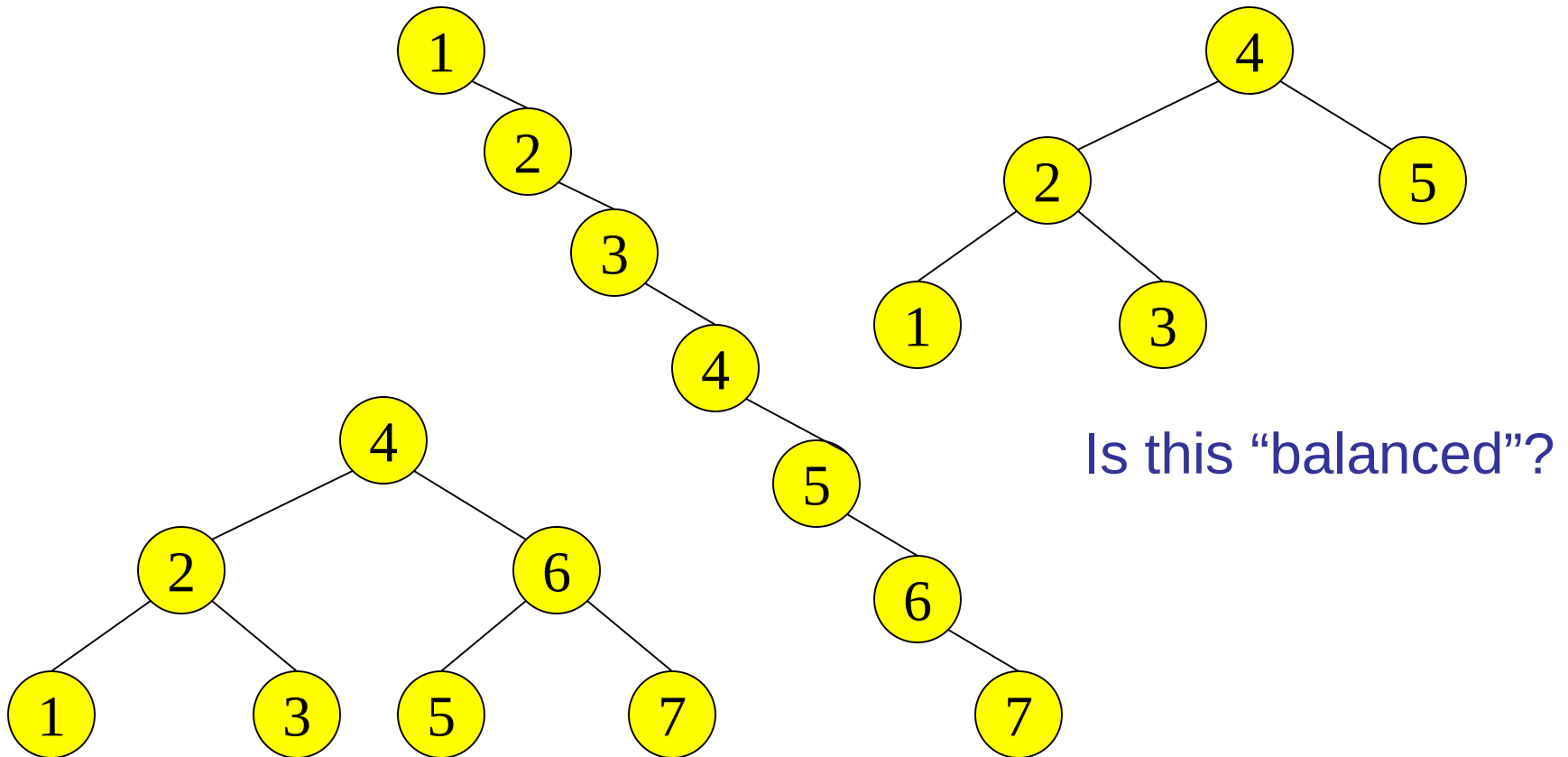


Algorithm Analysis And Data Structures

Prof. Dr. Ayla ŞAYLI

[http://avesis.yildiz.edu.tr/sayli/
sayli@yildiz.edu.tr](http://avesis.yildiz.edu.tr/sayli/sayli@yildiz.edu.tr)

Balanced and unbalanced BST



Approaches to balancing trees

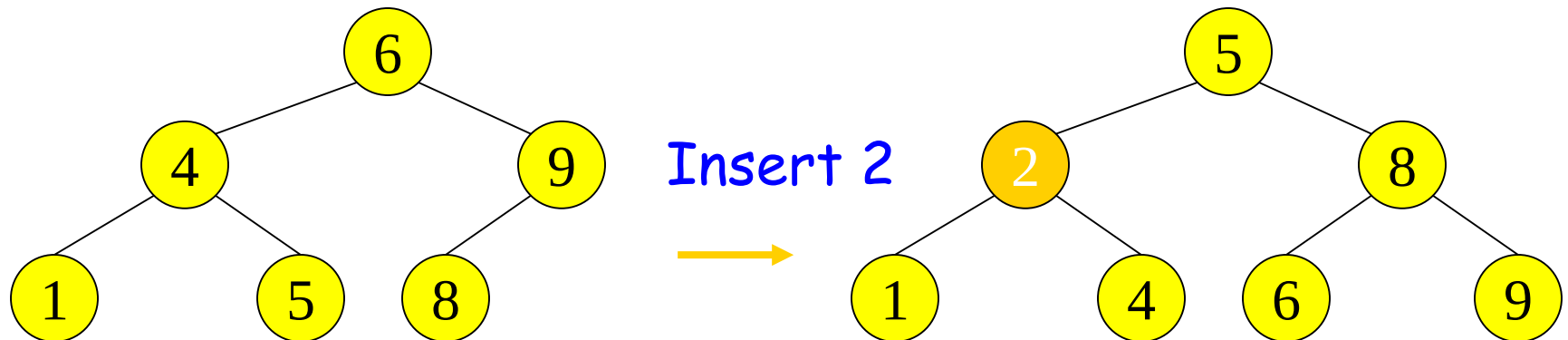
- Don't balance
 - ❖ May end up with some nodes very deep
- Strict balance
 - ❖ The tree must always be balanced perfectly
- Pretty good balance
 - ❖ Only allow a little out of balance
- Adjust on access
 - ❖ Self-adjusting

Balancing Binary Search Trees

- Many algorithms exist for keeping binary search trees balanced
 - ❖ Adelson-Velskii and Landis (**AVL**) trees (height-balanced trees)
 - ❖ **Splay trees** and other self-adjusting trees
 - ❖ **B-trees** and other multiway search trees

Perfect Balance

- Want a **complete tree** after every operation
 - ❖ tree is full except possibly in the lower right
- This is expensive
 - ❖ For example, insert 2 in the tree on the left and then rebuild as a complete tree



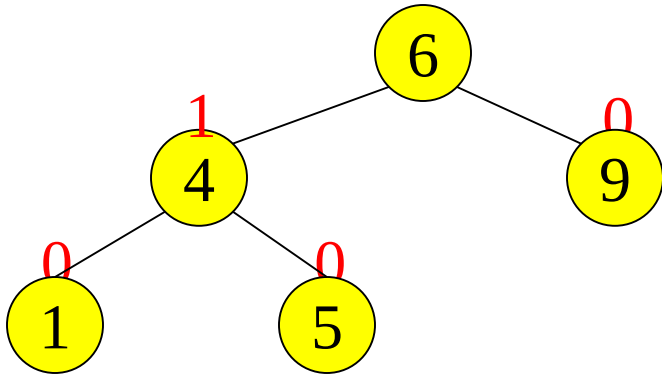
AVL - Good but not Perfect Balance

- AVL trees are height-balanced binary search trees
- Balance factor of a node
 - ❖ $\text{Height}(\text{left subtree}) - \text{height}(\text{right subtree})$
- An AVL tree has balance factor calculated at every node
 - ❖ For every node, heights of left and right subtree can differ by no more than 1
 - ❖ Store current heights in each node

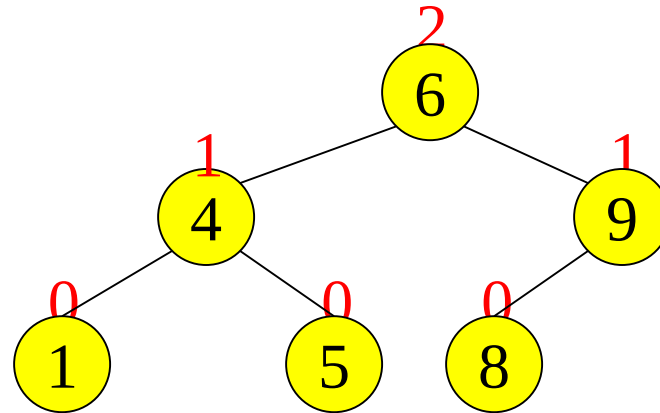
Node Heights

Tree A (AVL)

height=2 BF=1-0=1



Tree B (AVL)



height of node = h

balance factor = $h_{\text{left}} - h_{\text{right}}$

empty height = -1

Insertions in AVL Trees

Let the node that needs rebalancing be α .

There are 4 cases:

Outside Cases (require single rotation) :

1. Insertion into **left** subtree **of left** child of α .
2. Insertion into **right** subtree **of right** child of α .

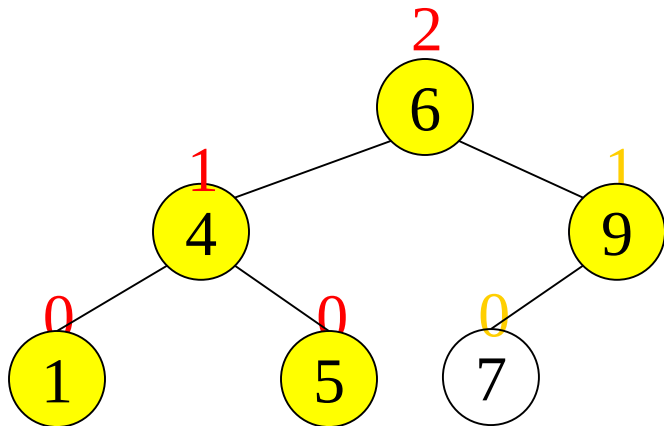
Inside Cases (require double rotation) :

3. Insertion into **right** subtree **of left** child of α .
4. Insertion into **left** subtree **of right** child of α .

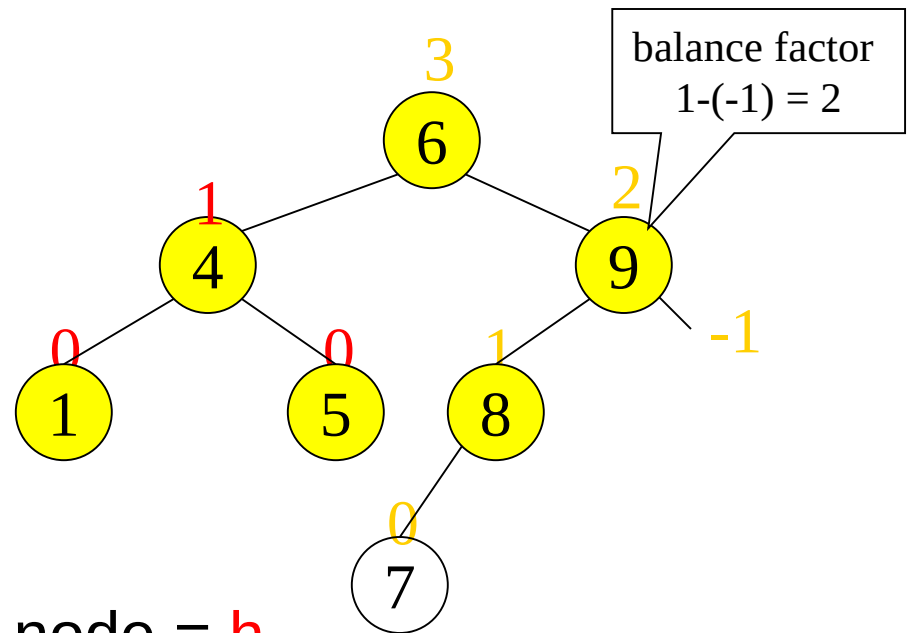
The rebalancing is performed through two separate rotation algorithms.

Node Heights after Insert 7

Tree A (AVL)



Tree B (not AVL)

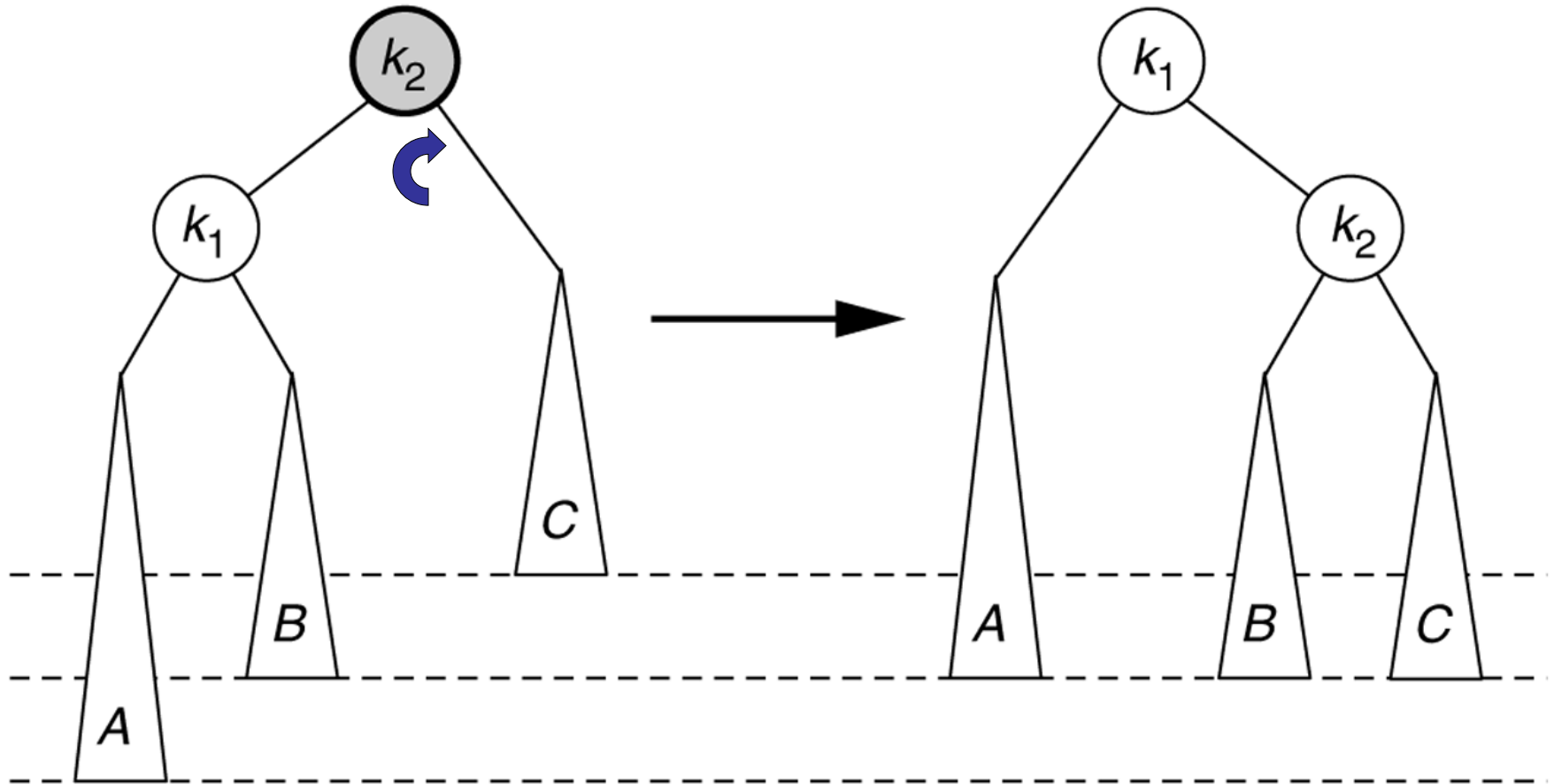


height of node = h
balance factor = $h_{\text{left}} - h_{\text{right}}$
empty height = -1

Insert and Rotation in AVL Trees

- Insert operation may cause balance factor to become 2 or -2 for some node
 - ❖ only nodes on the path from insertion point to root node have possibly changed in height
 - ❖ So after the Insert, go back up to the root node by node, updating heights
 - ❖ If a new balance factor (the difference $h_{\text{left}} - h_{\text{right}}$) is 2 or -2, adjust tree by *rotation* around the node

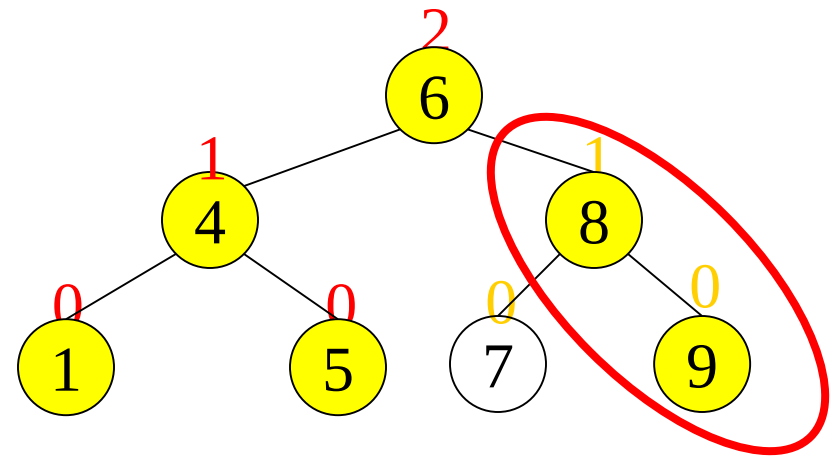
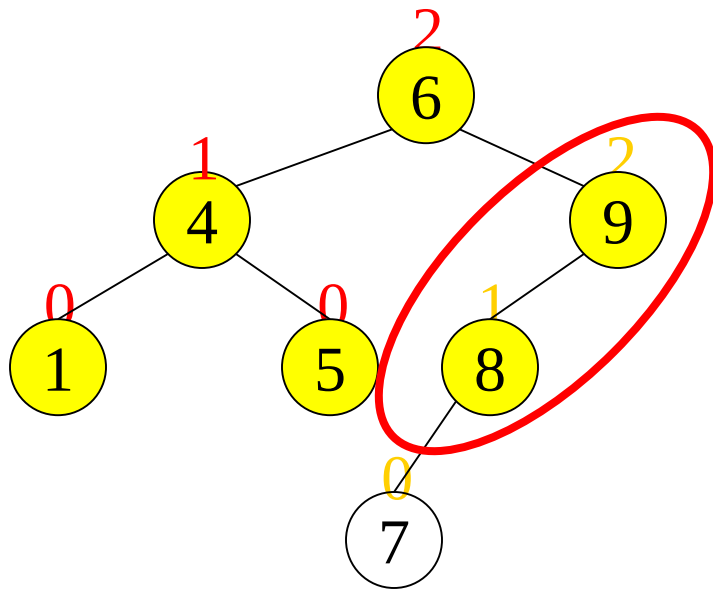
Single rotation to fix case 1: Rotate right



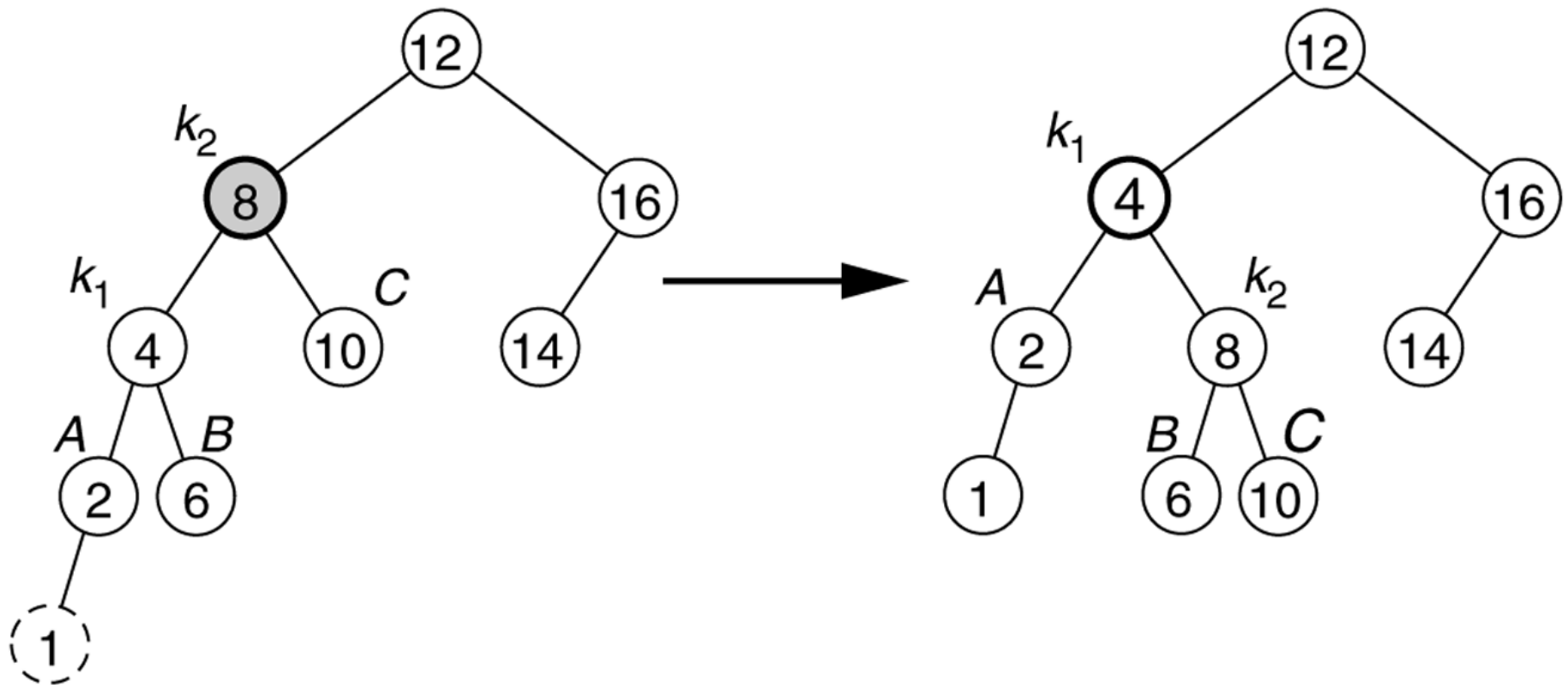
(a) Before rotation

(b) After rotation

Single Rotation in an AVL Tree



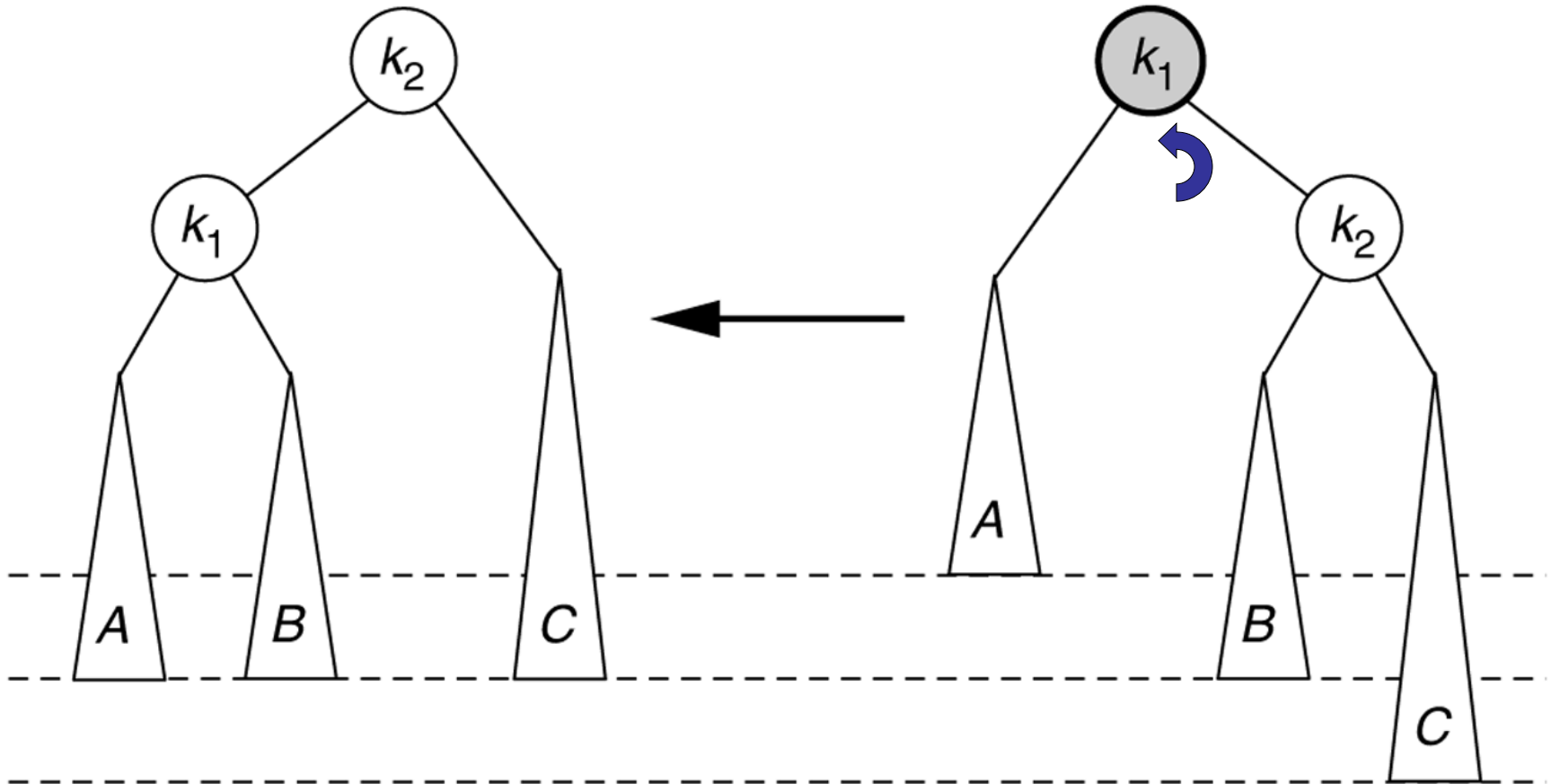
Single rotation fixes an AVL tree after insertion of 1.



(a) Before rotation

(b) After rotation

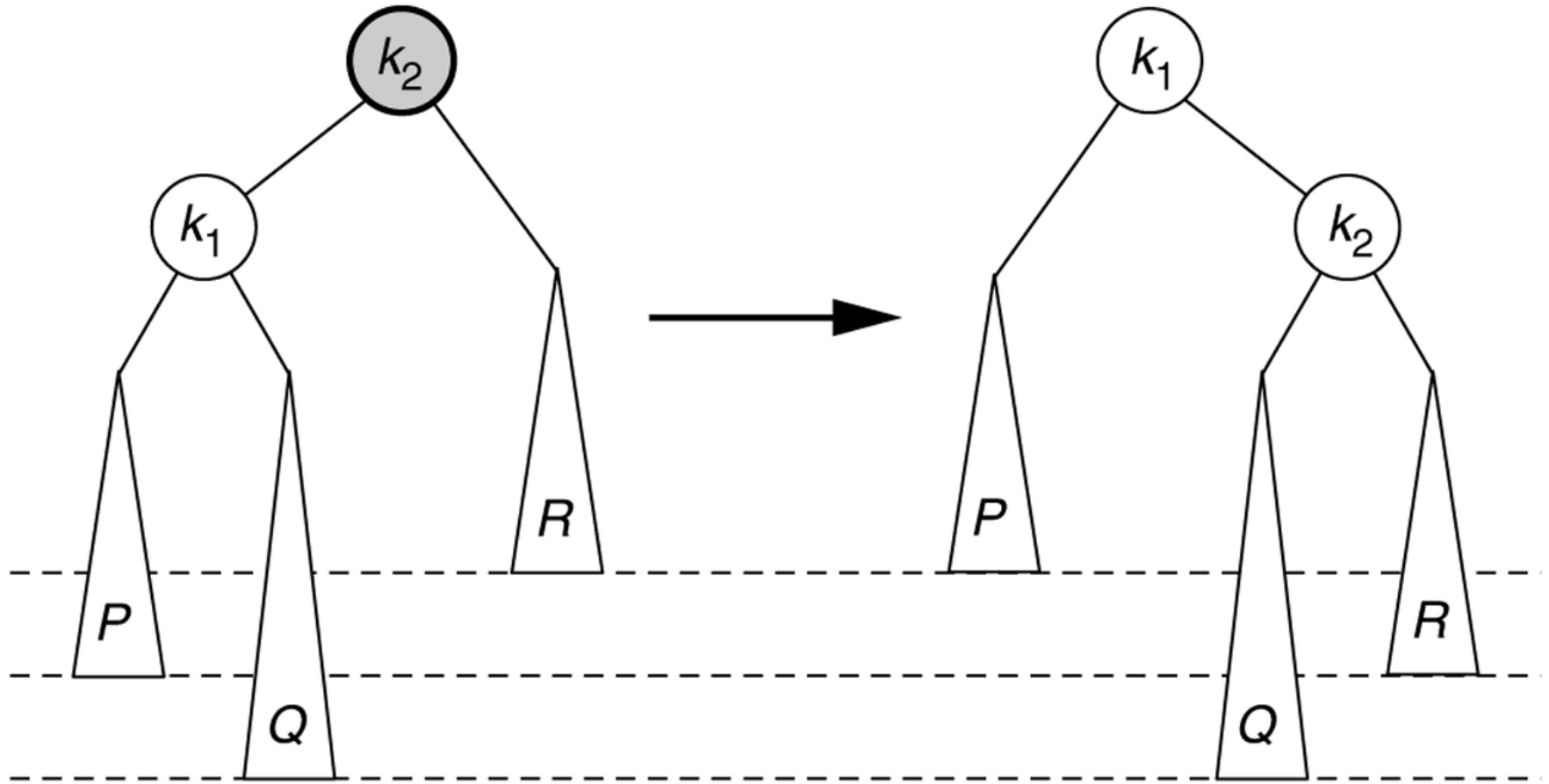
Symmetric single rotation to fix case 4 : Rotate left



(a) After rotation

(b) Before rotation

Single rotation does not fix case 2.



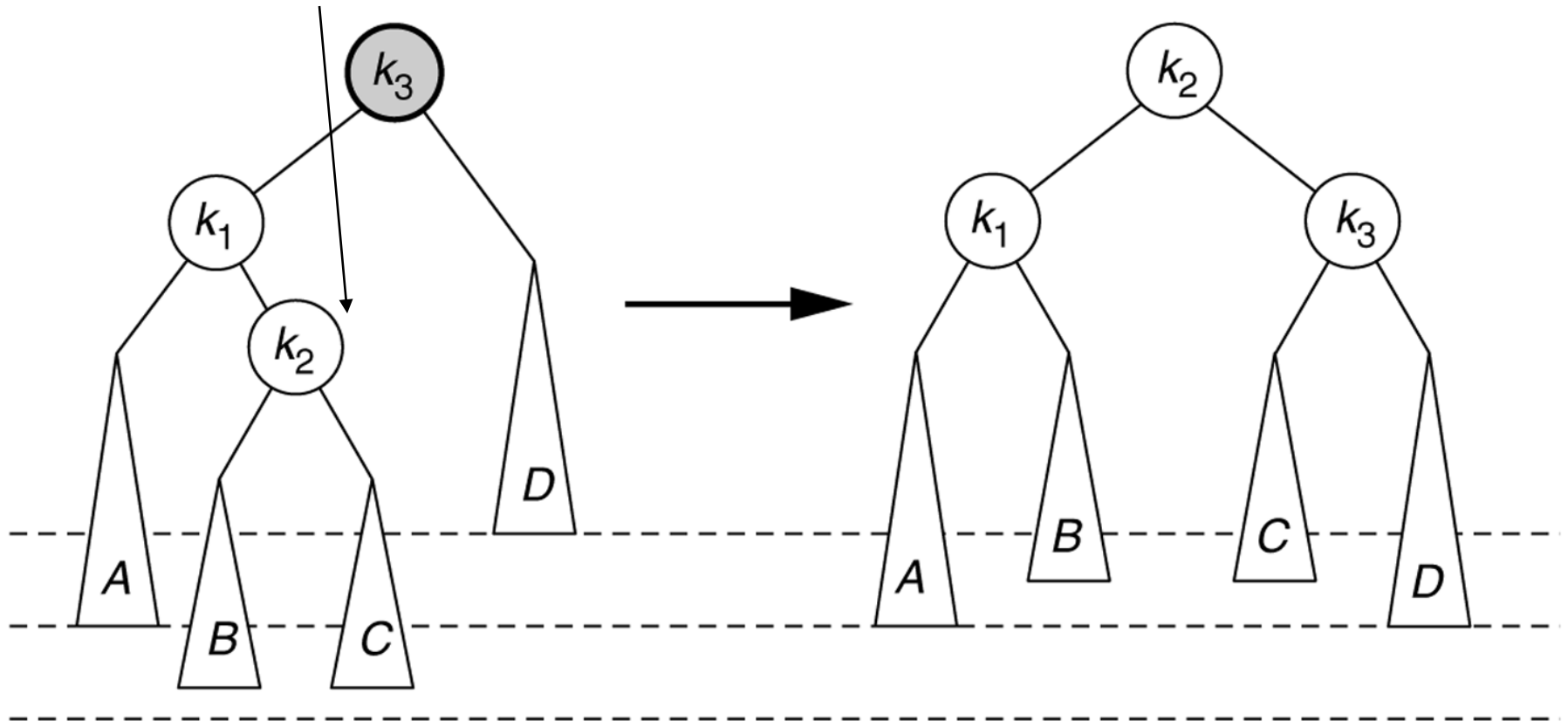
(a) Before rotation

(b) After rotation

Left-right double rotation to fix case 2

Lift this up:

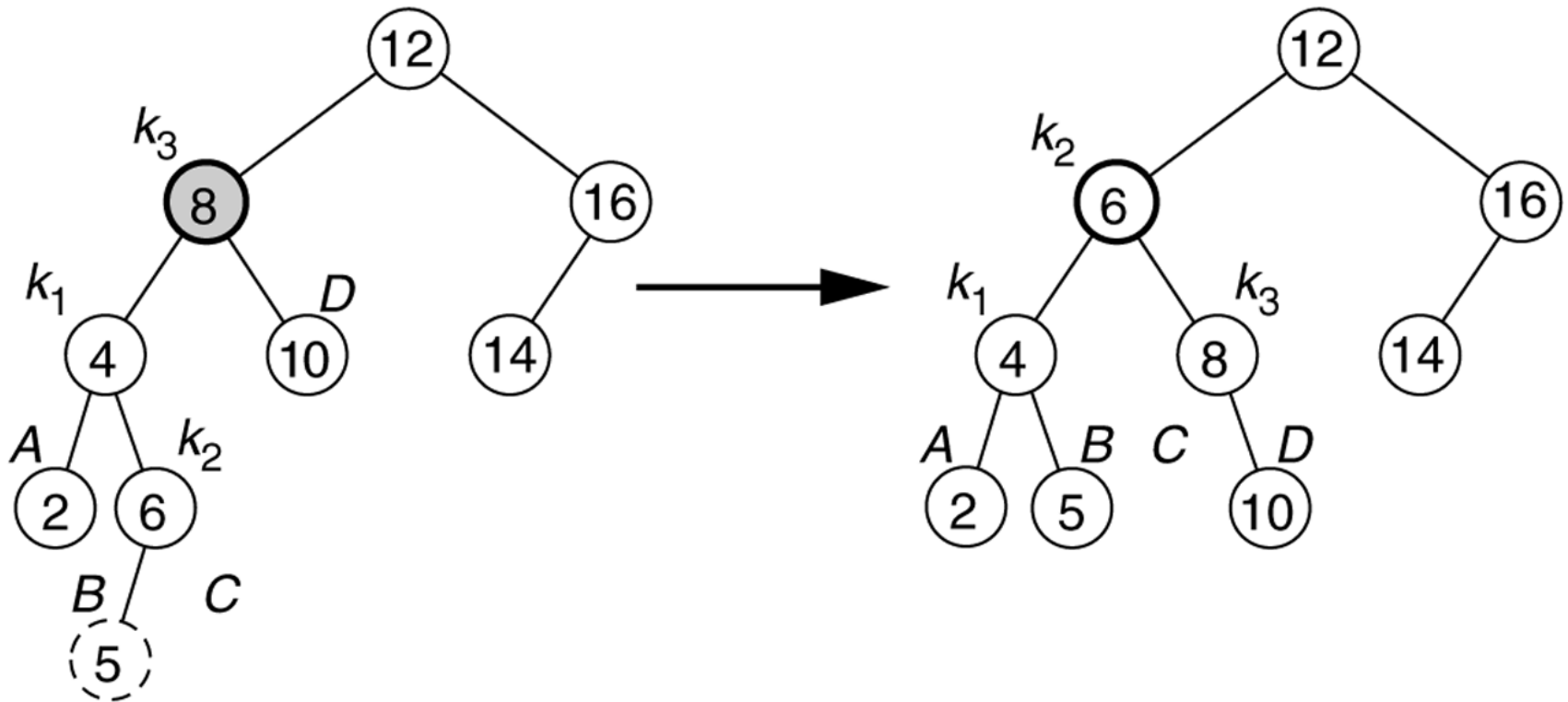
*first rotate left between (k_1, k_2) ,
then rotate right between (k_3, k_2)*



(a) Before rotation

(b) After rotation

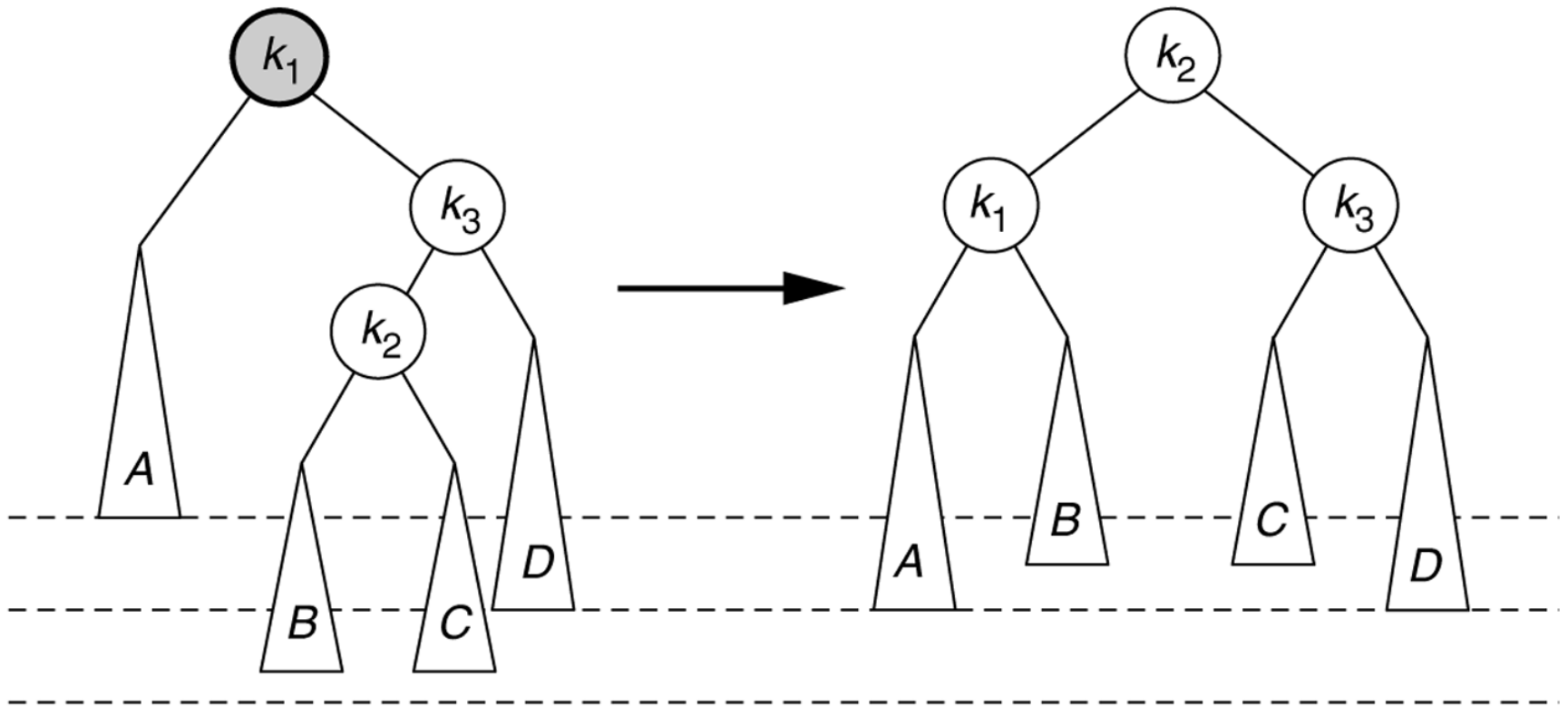
Double rotation fixes AVL tree after the insertion of 5.



(a) Before rotation

(b) After rotation

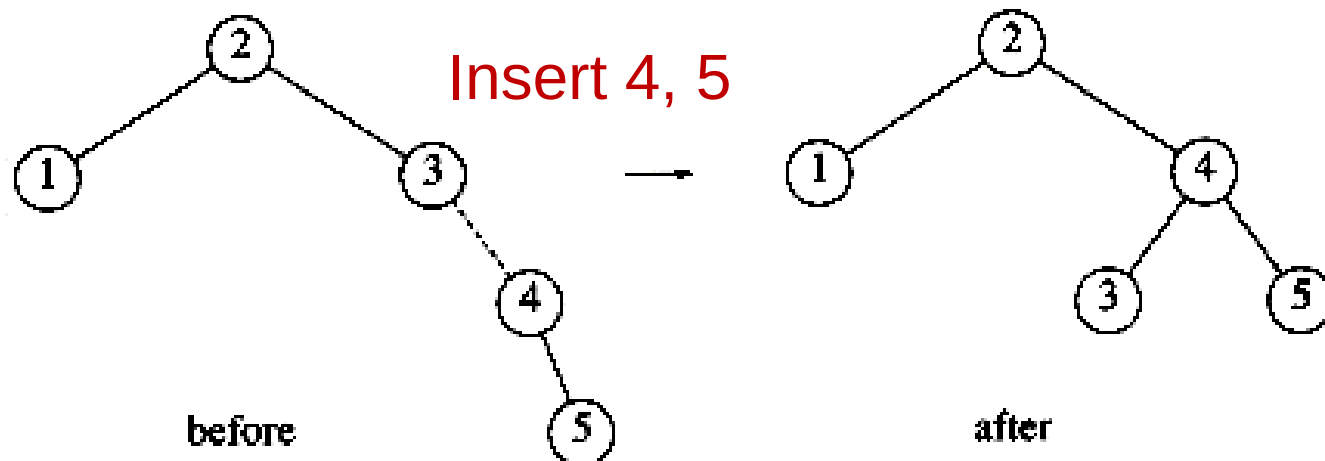
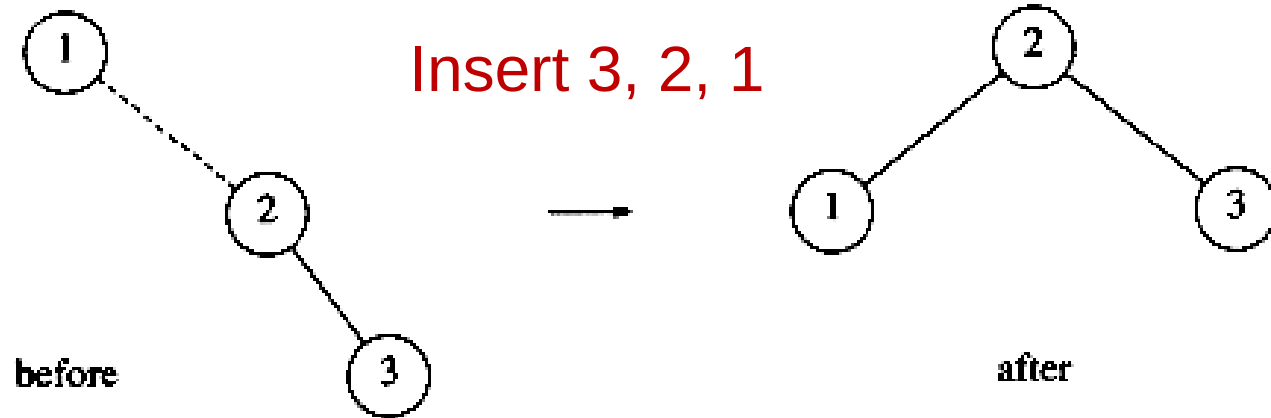
Right-Left double rotation to fix case 3.



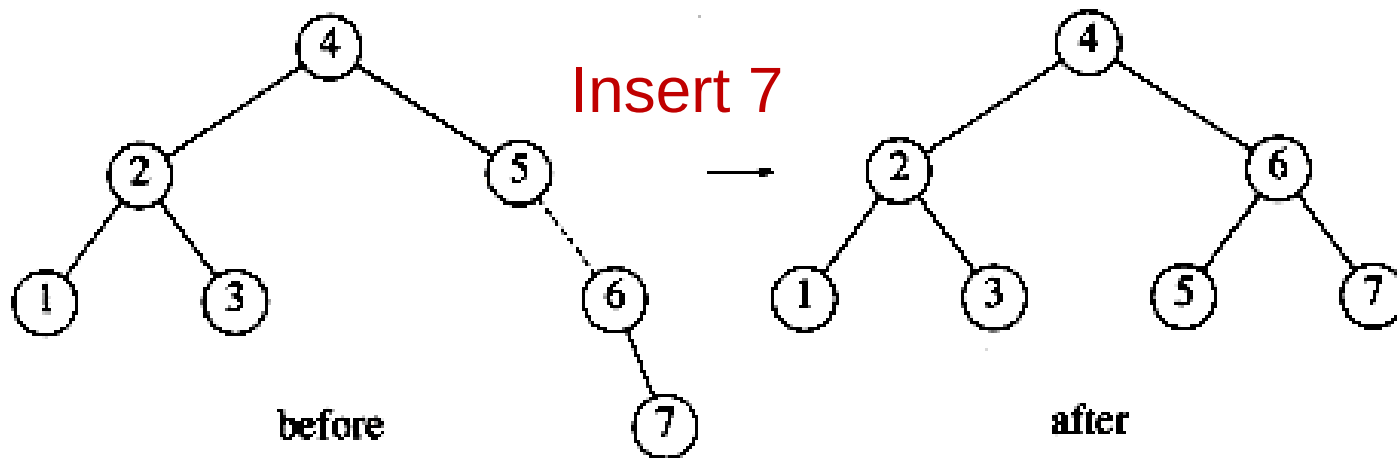
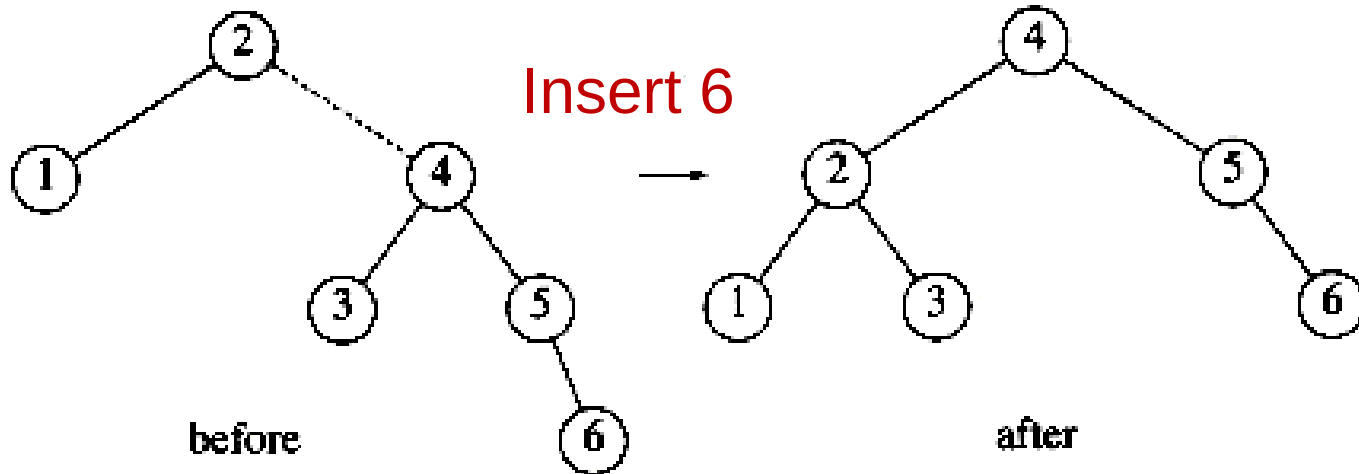
(a) Before rotation

(b) After rotation

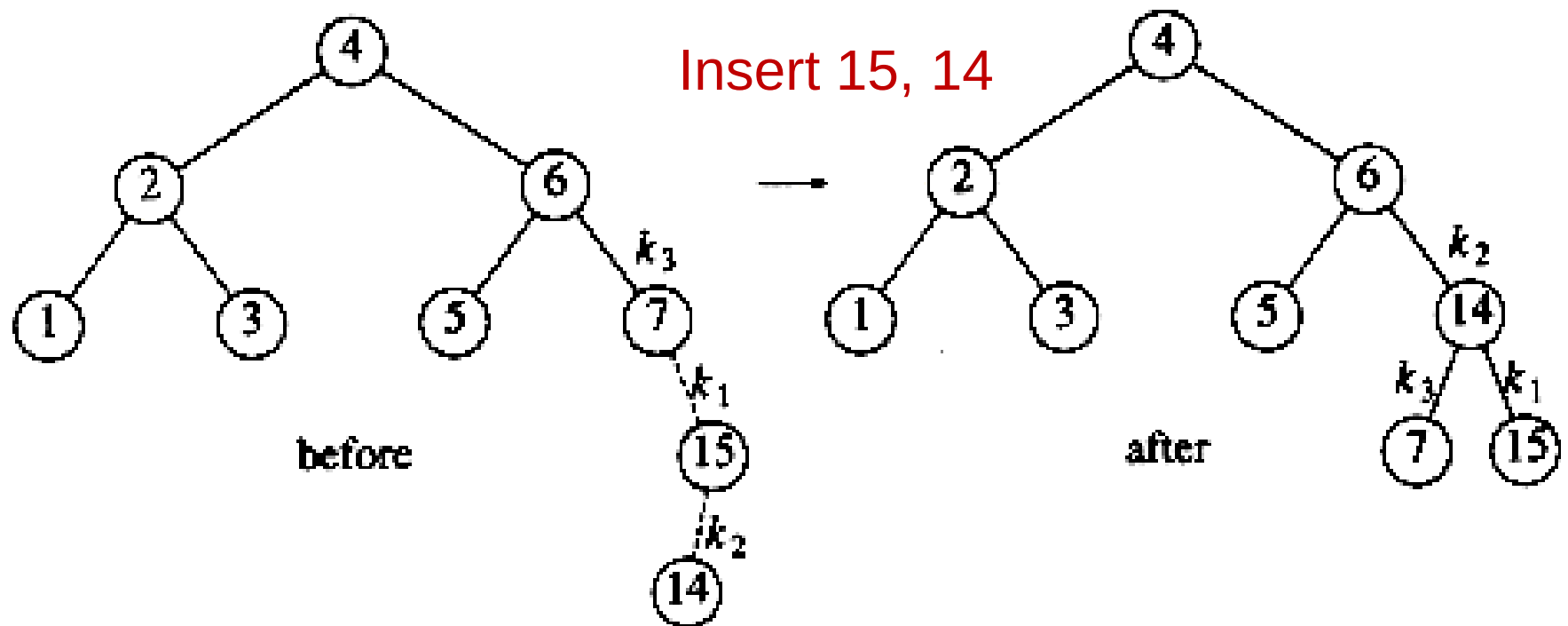
Example 1: Start with an empty AVL tree and insert the items 3, 2, 1, 4, 5, 6, 7, 15, 14, 13, 12, 11, 10, 9, 8 and 8.5 in sequential order



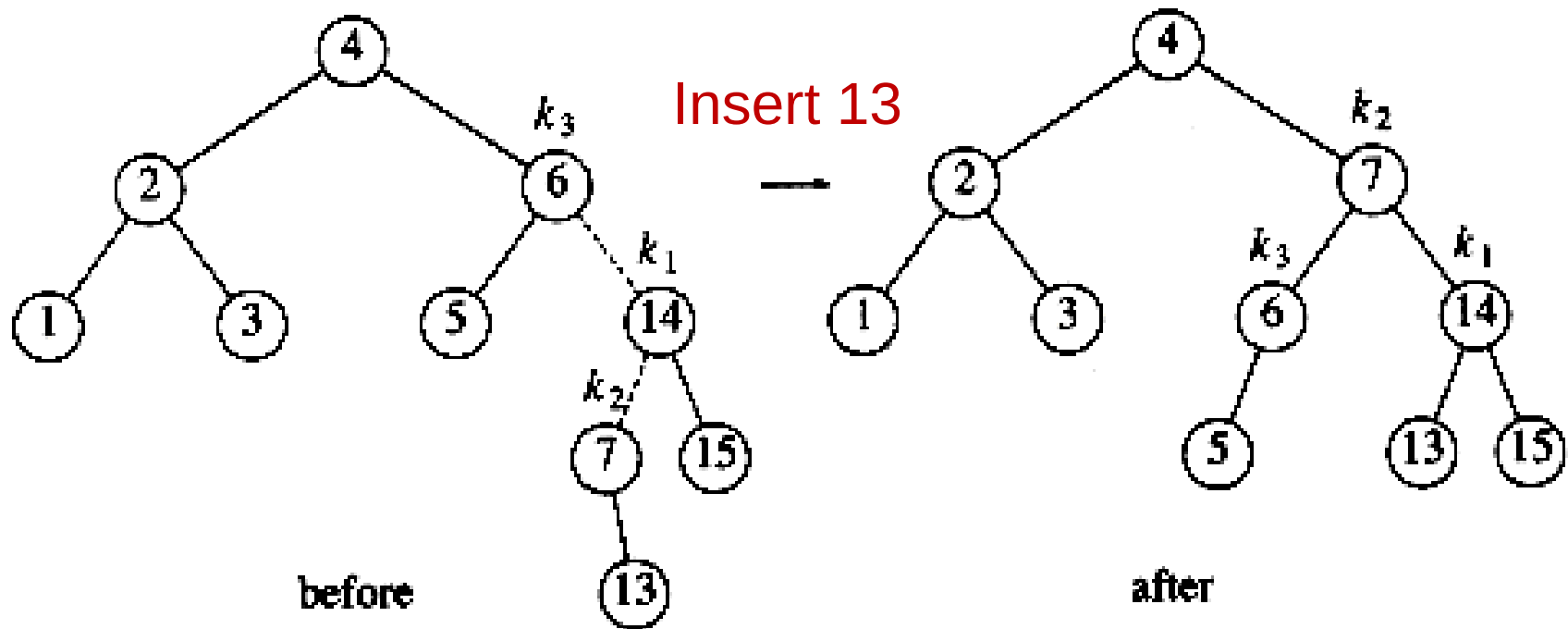
Example: Start with an empty AVL tree and insert the items 3,2,1,4,5,6,7,15,14, 13, 12,11, 10,9, 8 and 8.5 in sequential order



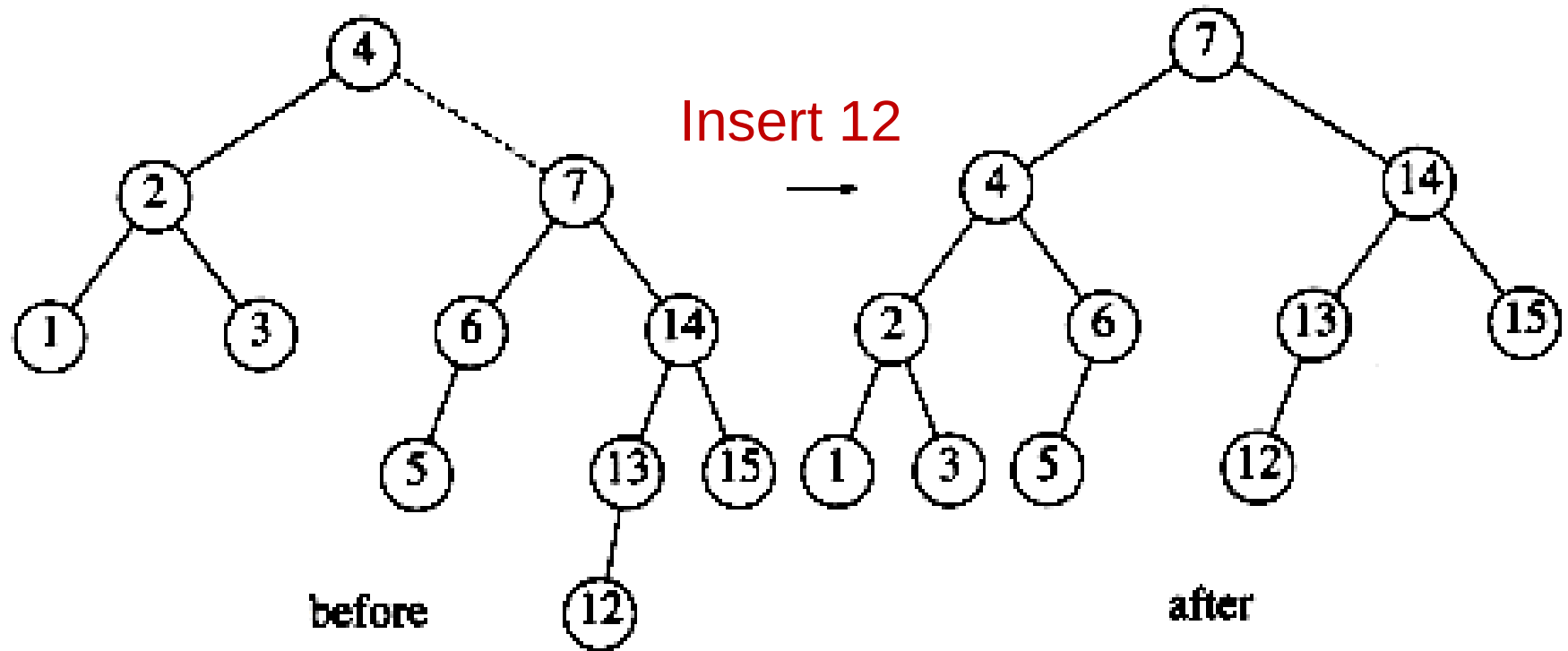
Example: Start with an empty AVL tree and insert the items 3, 2, 1, 4, 5, 6, 7, 15, 14, 13, 12, 11, 10, 9, 8 and 8.5 in sequential order



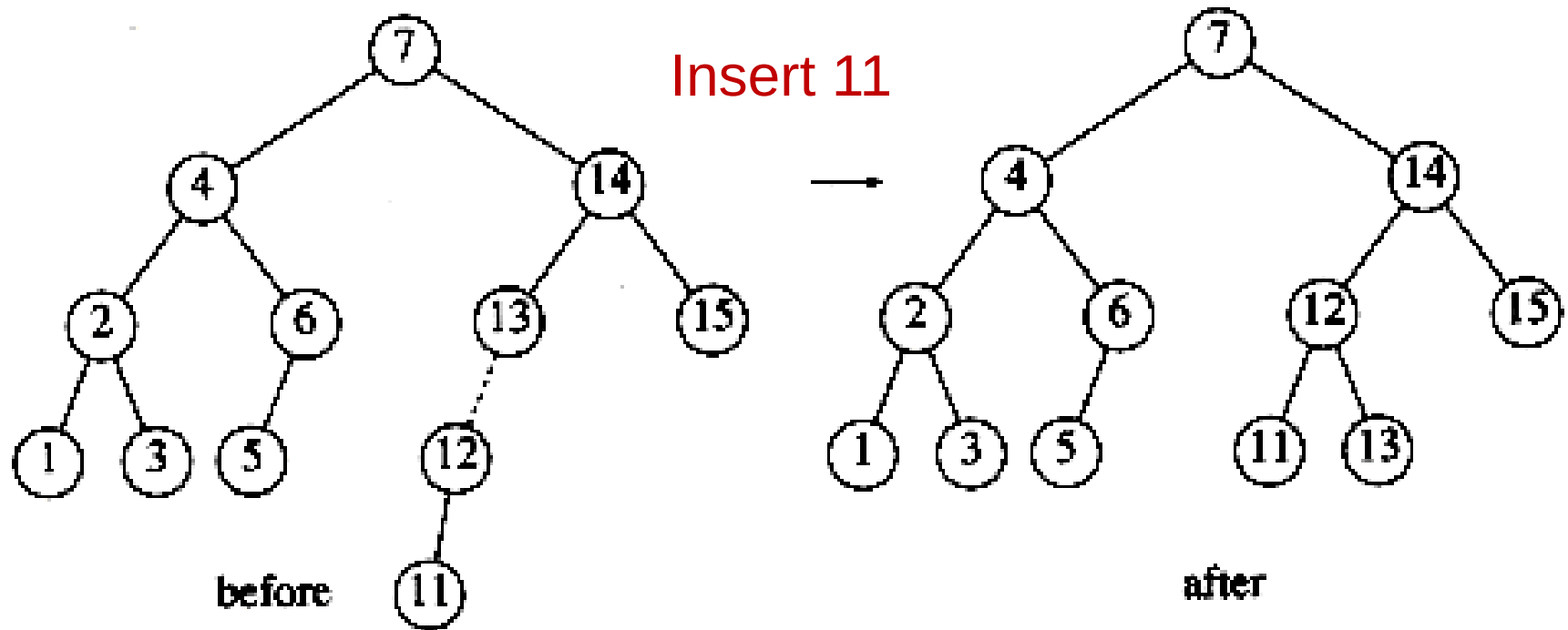
Example: Start with an empty AVL tree and insert the items 3, 2, 1, 4, 5, 6, 7, 15, 14, 13, 12, 11, 10, 9, 8 and 8.5 in sequential order



Example: Start with an empty AVL tree and insert the items 3,2,1,4,5,6,7,15,14, 13, 12,11, 10,9, 8 and 8.5 in sequential order

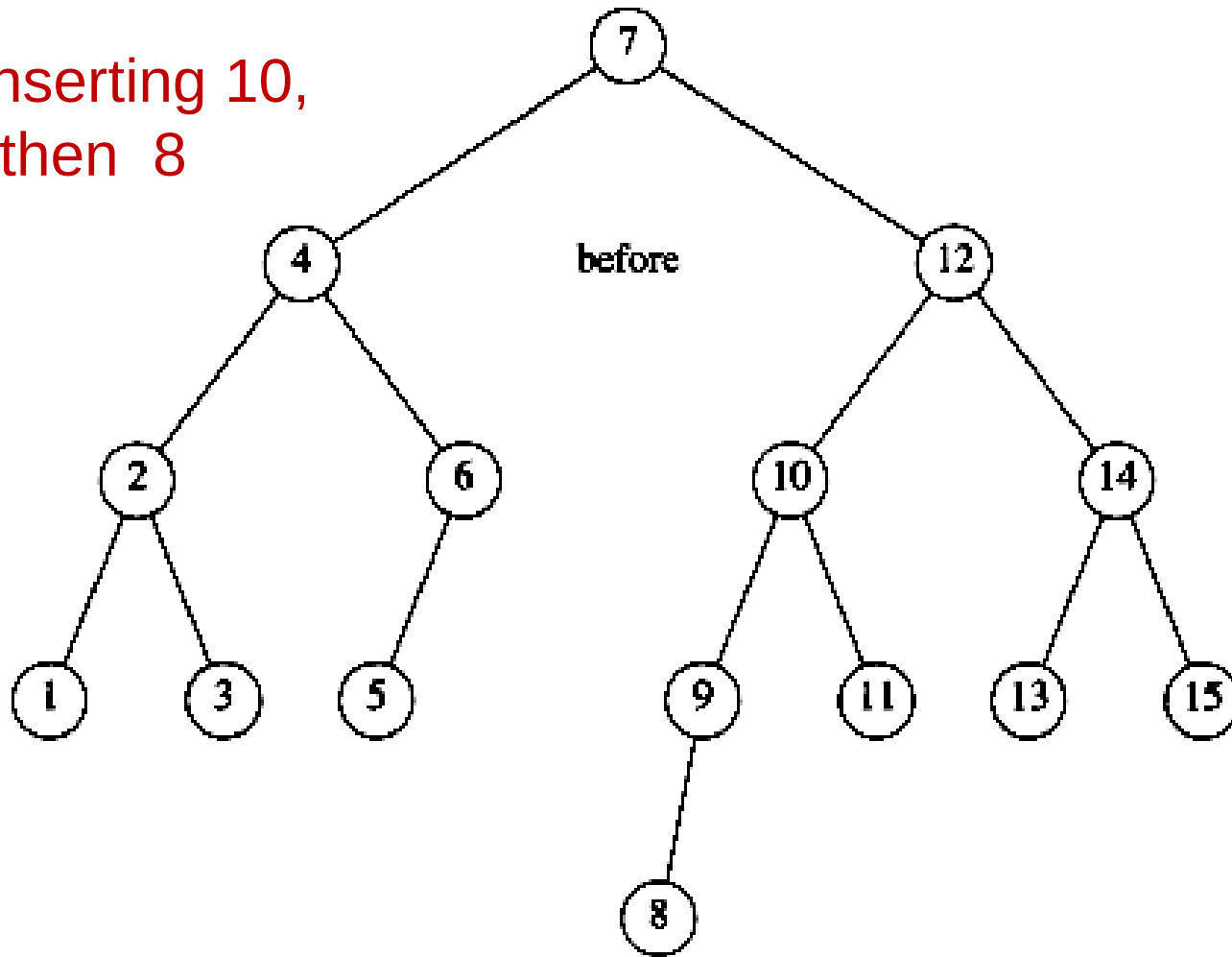


Example: Start with an empty AVL tree and insert the items 3,2,1,4,5,6,7,15,14, 13, 12,11, 10,9, 8 and 8.5 in sequential order



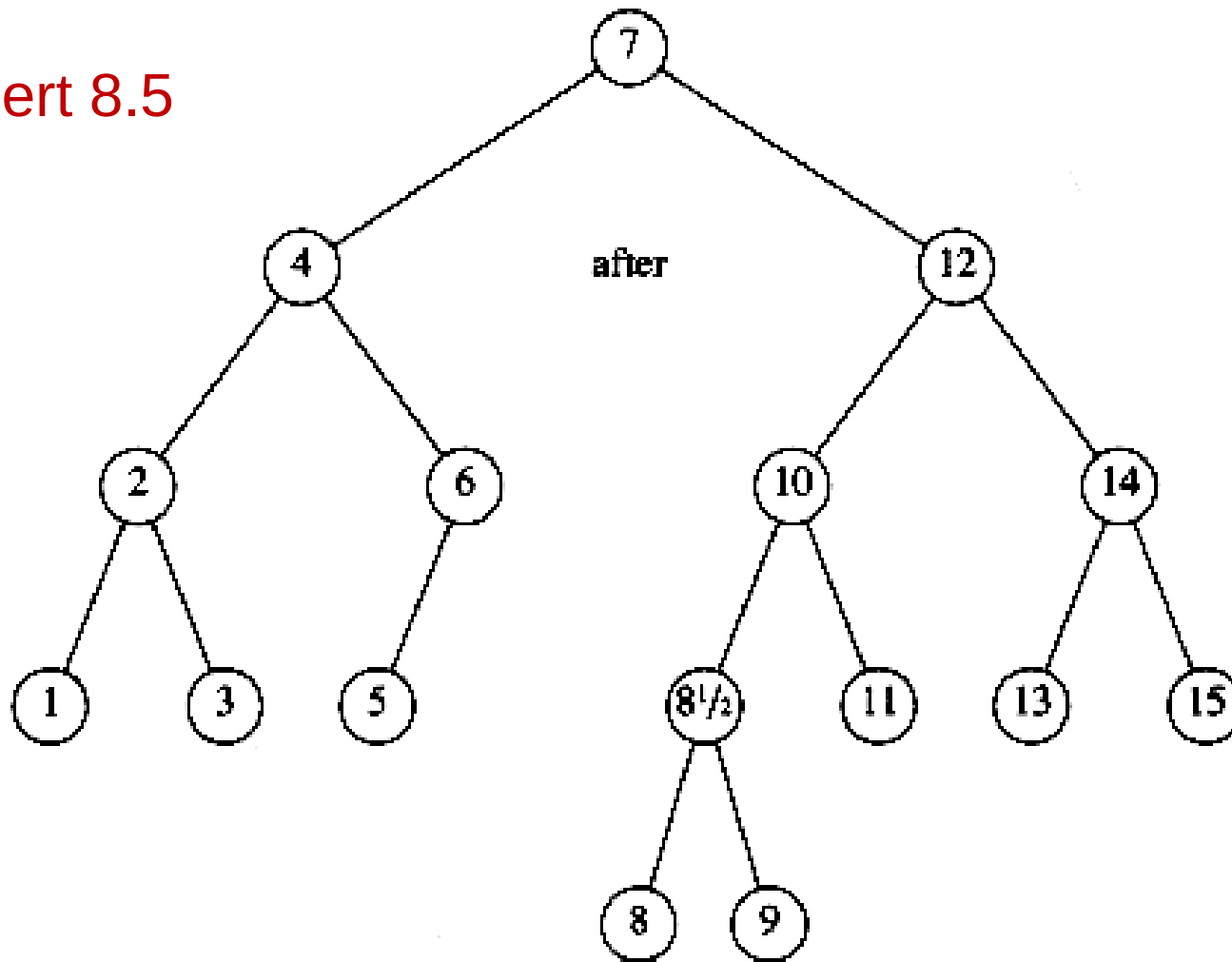
Example: Start with an empty AVL tree and insert the items 3,2,1,4,5,6,7,15,14, 13, 12,11, 10,9, 8 and 8.5 in sequential order

After inserting 10,
9 and then 8



Example: Start with an empty AVL tree and insert the items 3, 2, 1, 4, 5, 6, 7, 15, 14, 13, 12, 11, 10, 9, 8 and 8.5 in sequential order

Insert 8.5



AVL (Adelson-Velskii and Landis)

```
typedef struct avl_node *avl_ptr;
```

```
struct avl_node
```

```
{
```

```
    element_type element;
```

```
    avl_ptr left;
```

```
    avl_ptr right;
```

```
    int height;
```

```
};
```

```
typedef avl_ptr SEARCH_TREE;
```

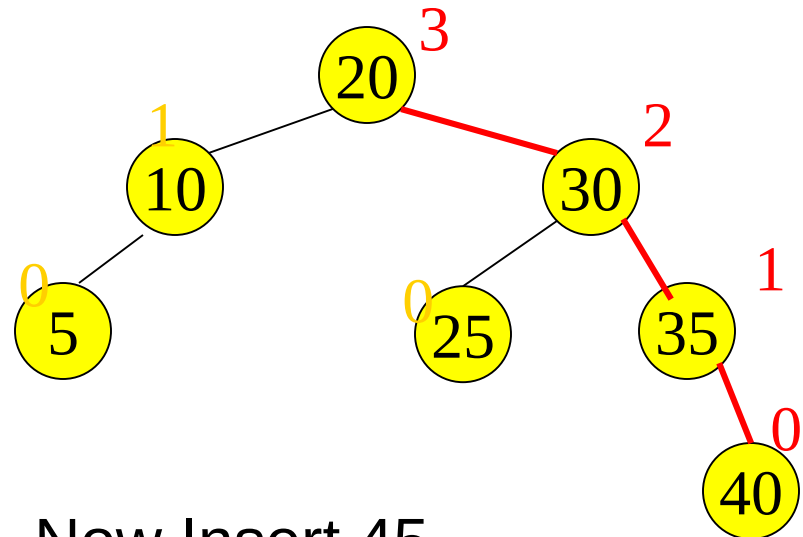
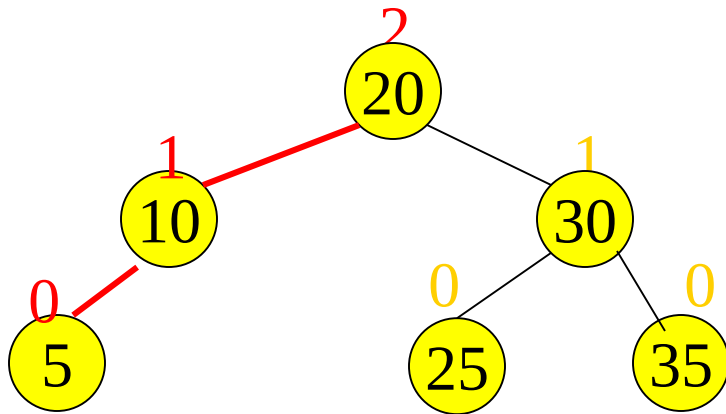
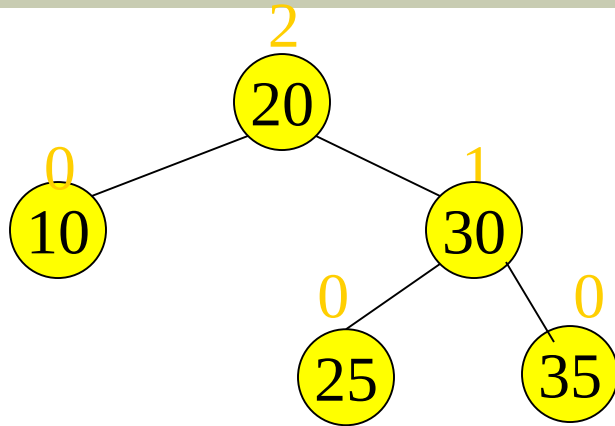
AVL (Adelson-Velskii and Landis)

```
int height( avl_ptr p )  
{  
    if( p == NULL )  
        return -1;  
    else  
        return p->height;  
}
```

Figure 4.36 Function to compute height of an AVL node

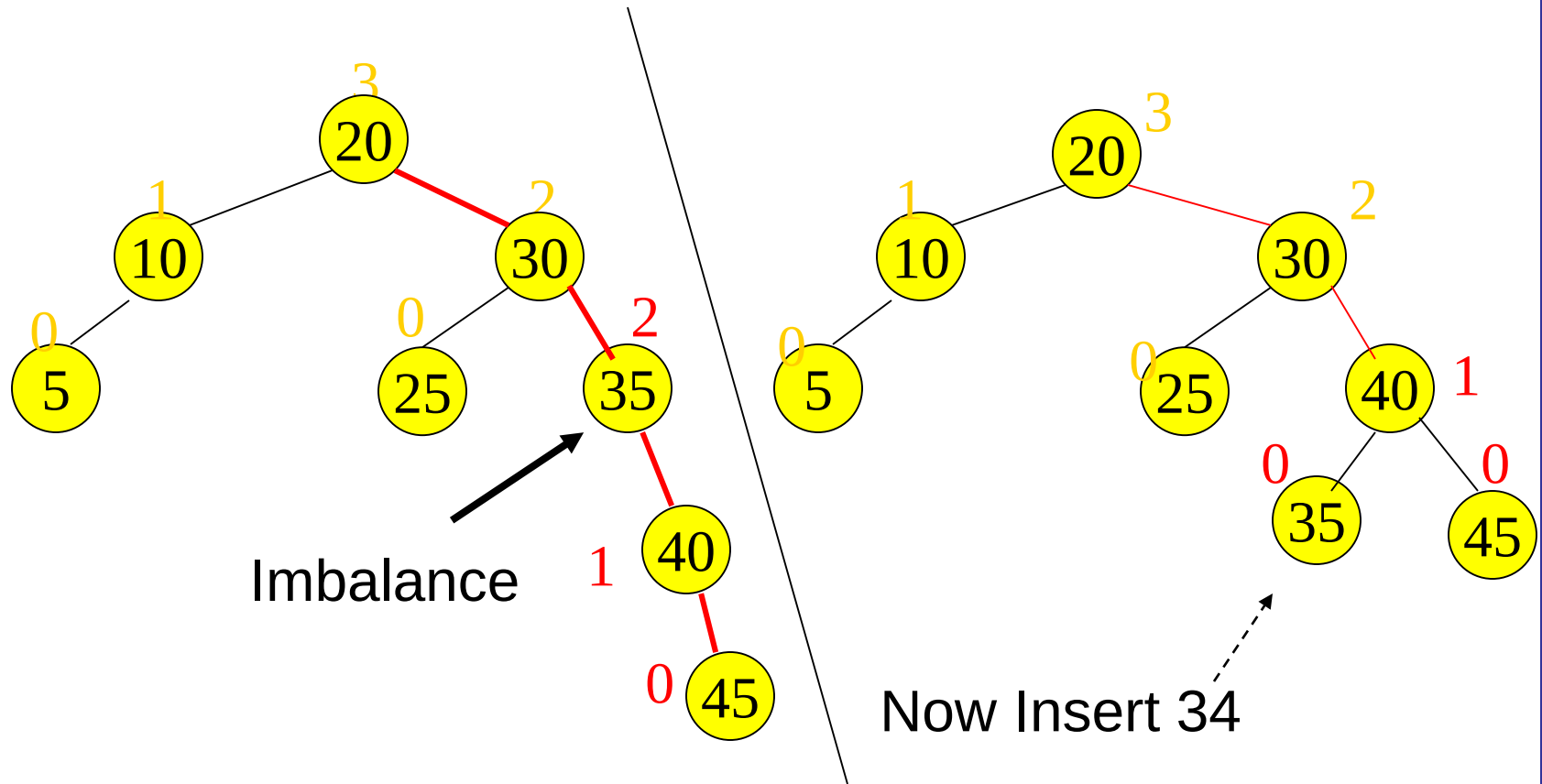
Example 2

Insert 5

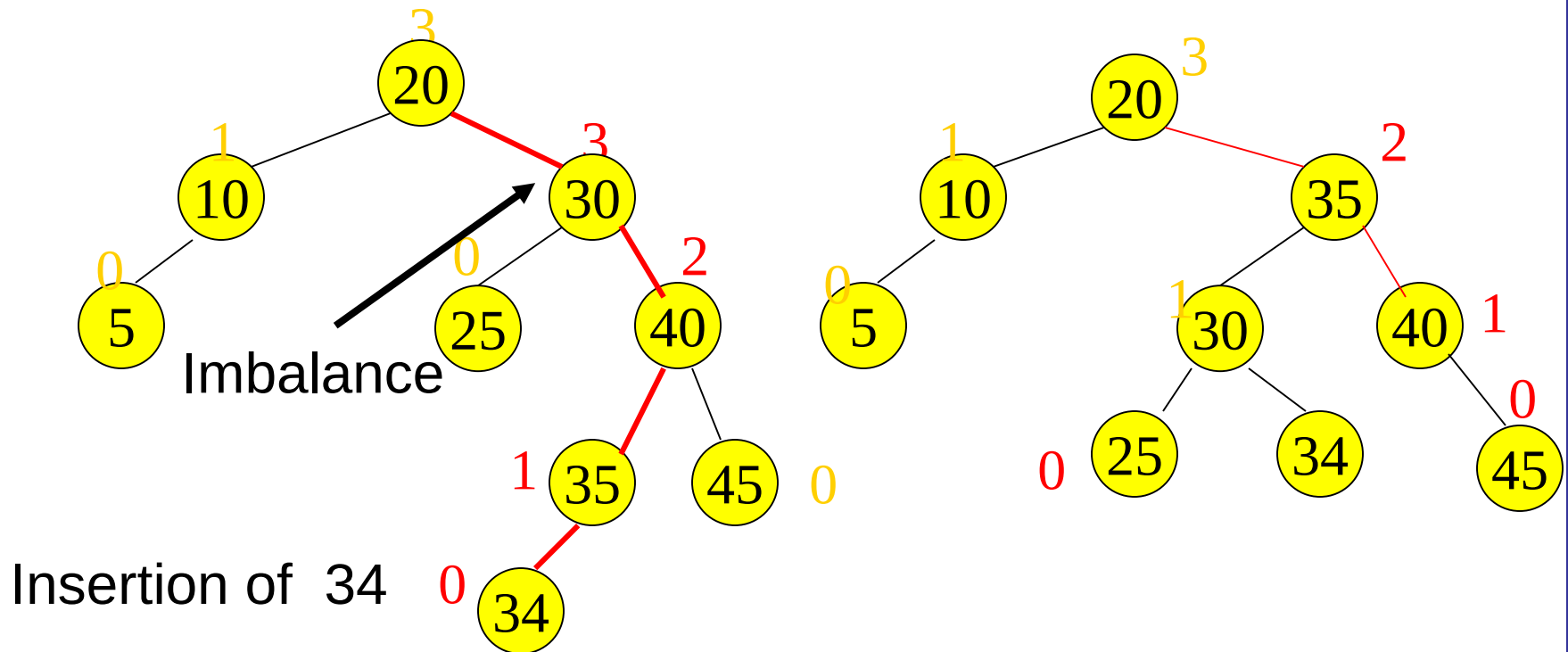


Now Insert 45

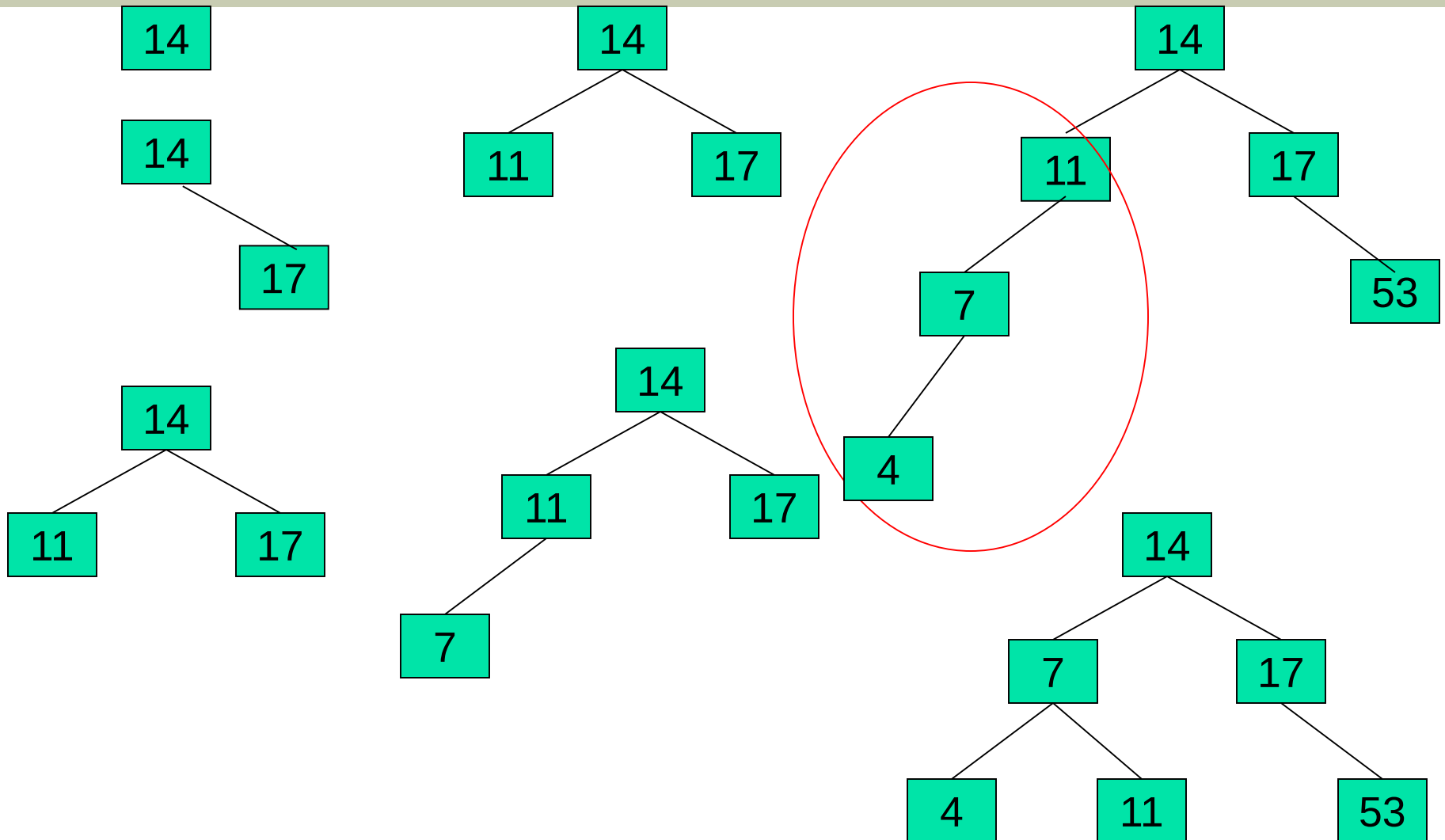
Single rotation (outside case)



Double rotation (inside case)

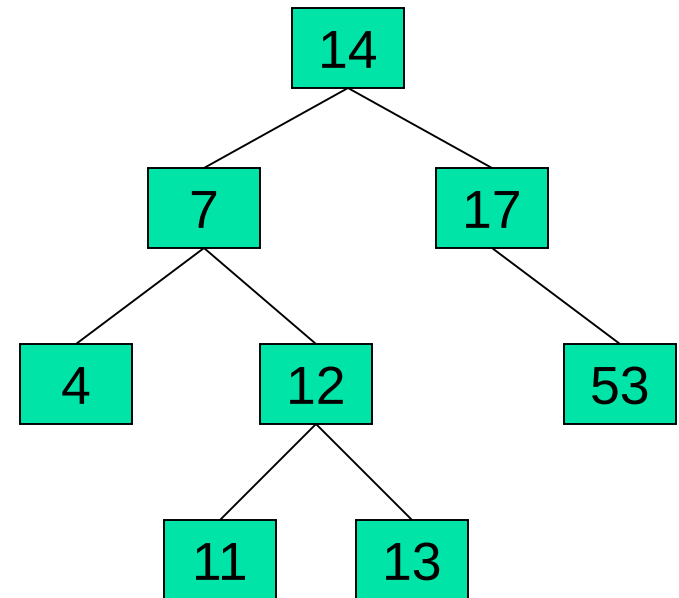
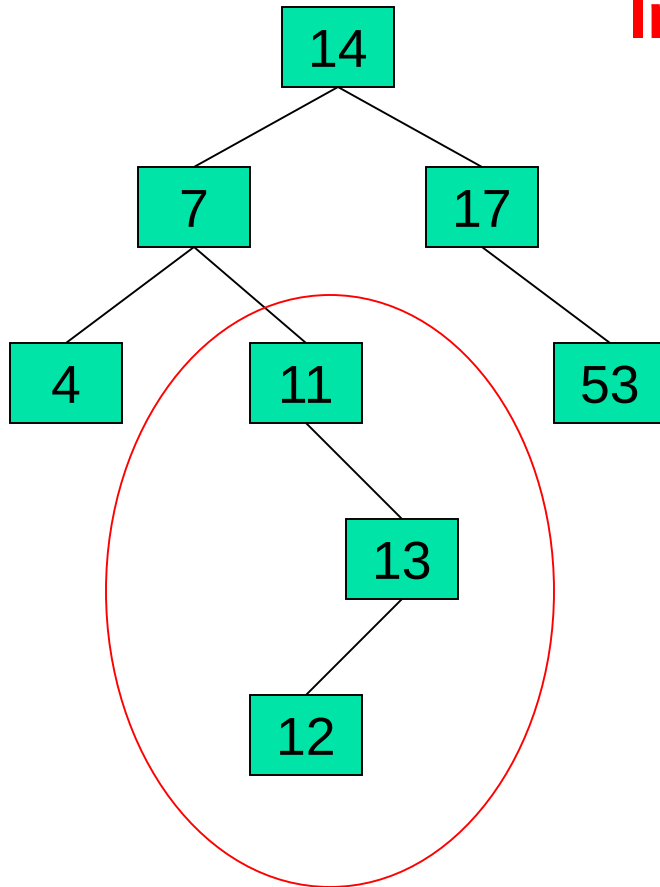


Example 3: Insert 14, 17, 11, 7, 53, 4, 13 into an empty AVL tree

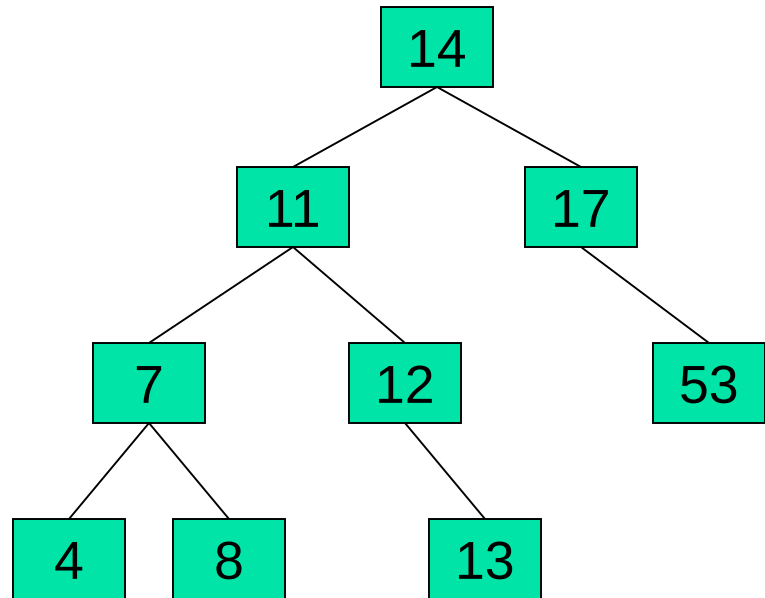
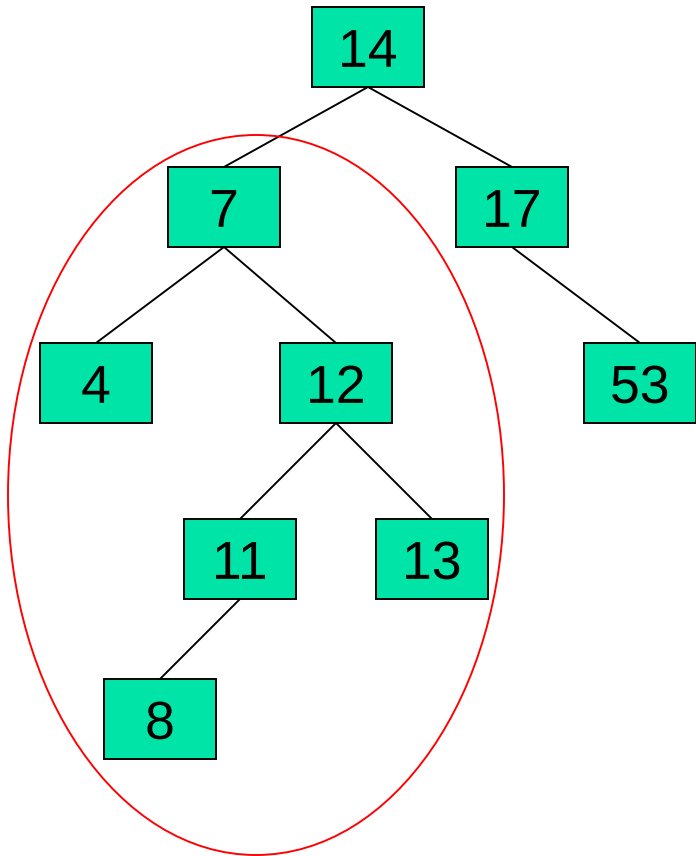


AVL Tree Example:

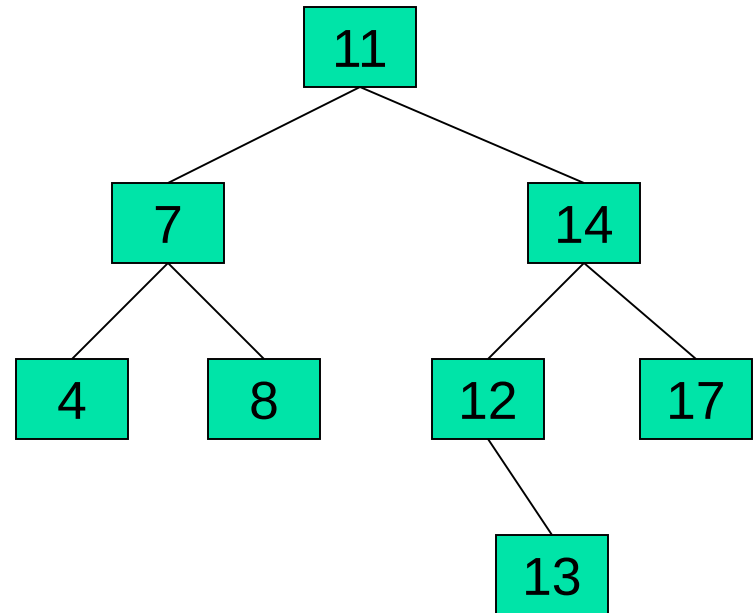
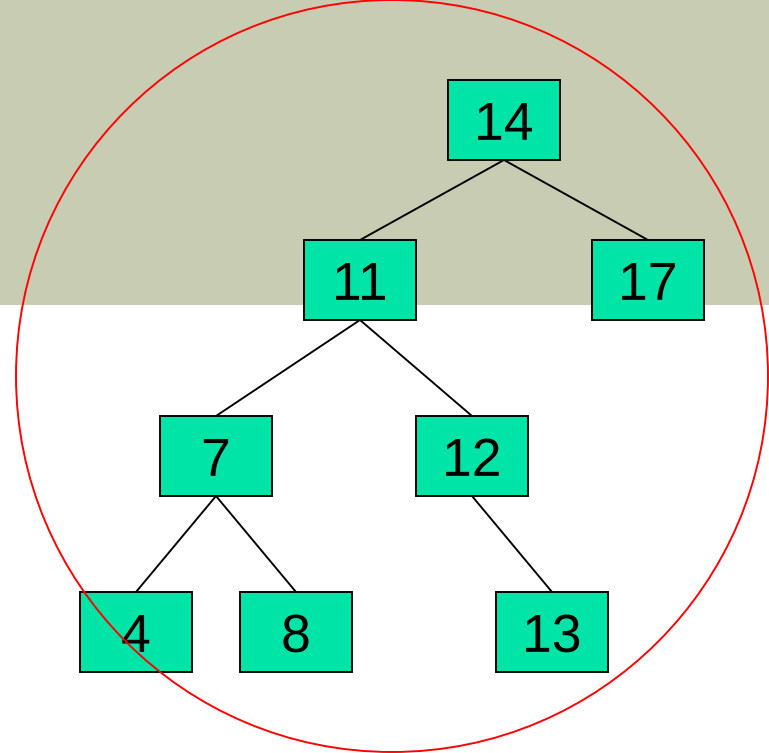
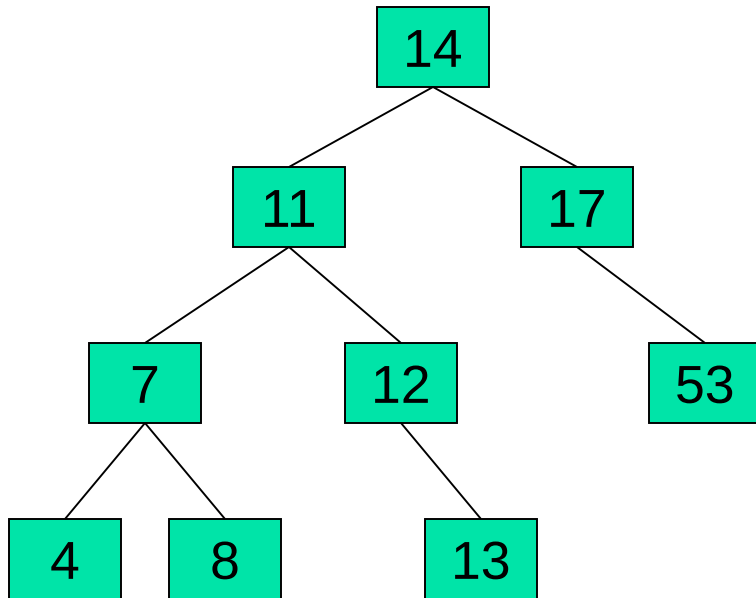
Insert 12



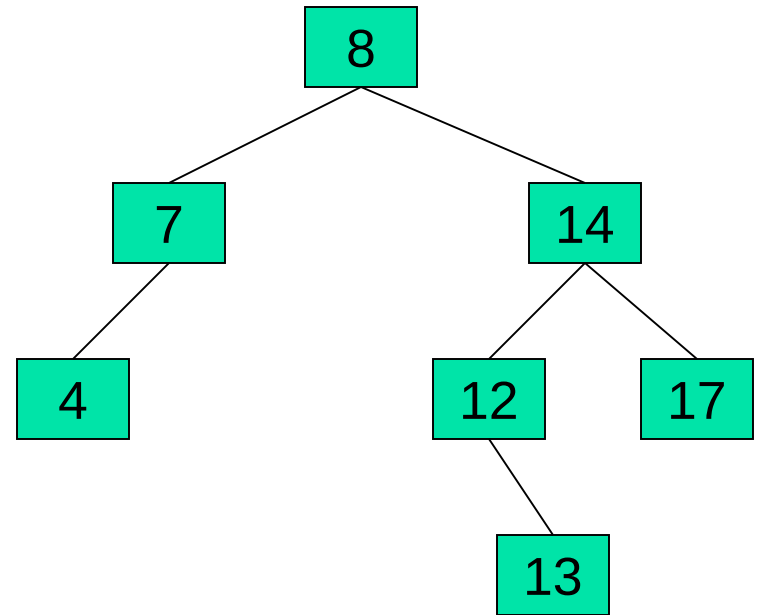
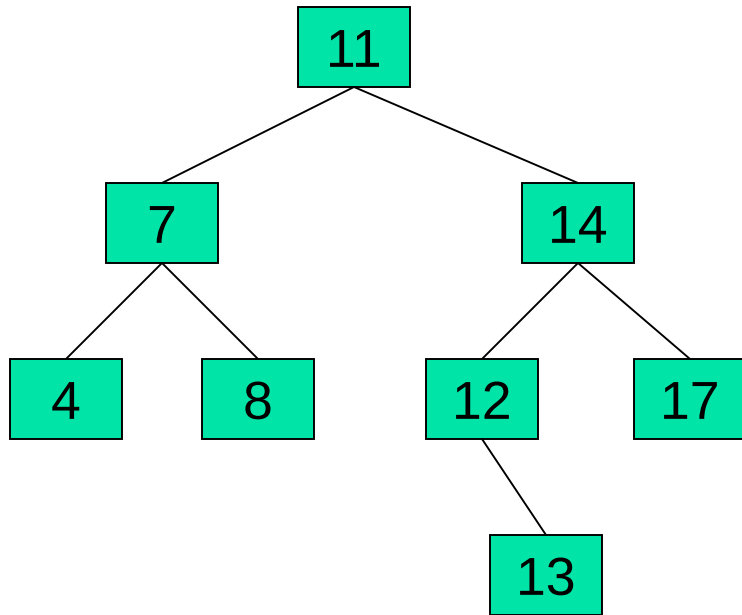
Insert 8



Remove 53

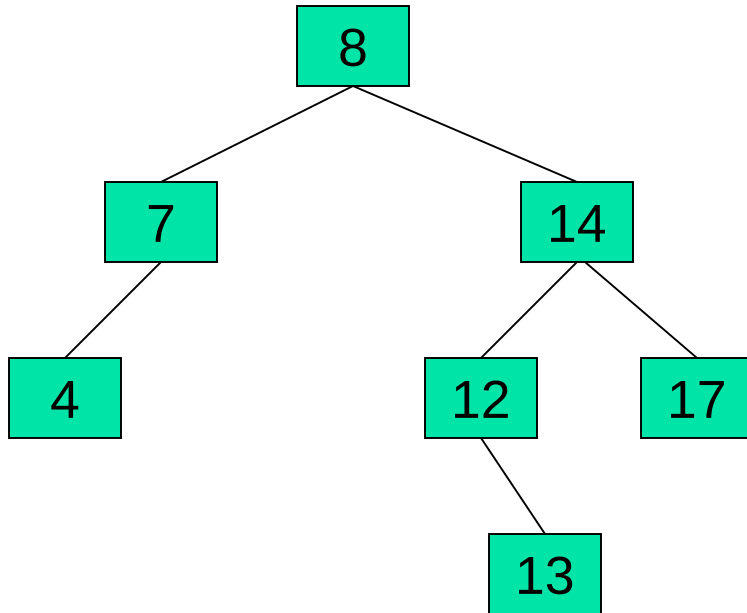


Remove 11

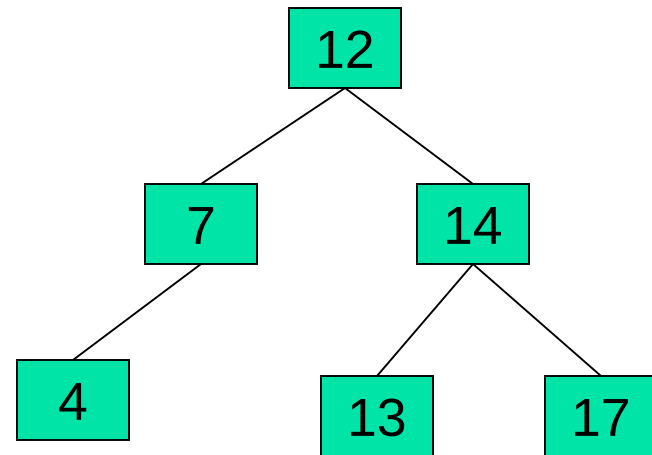
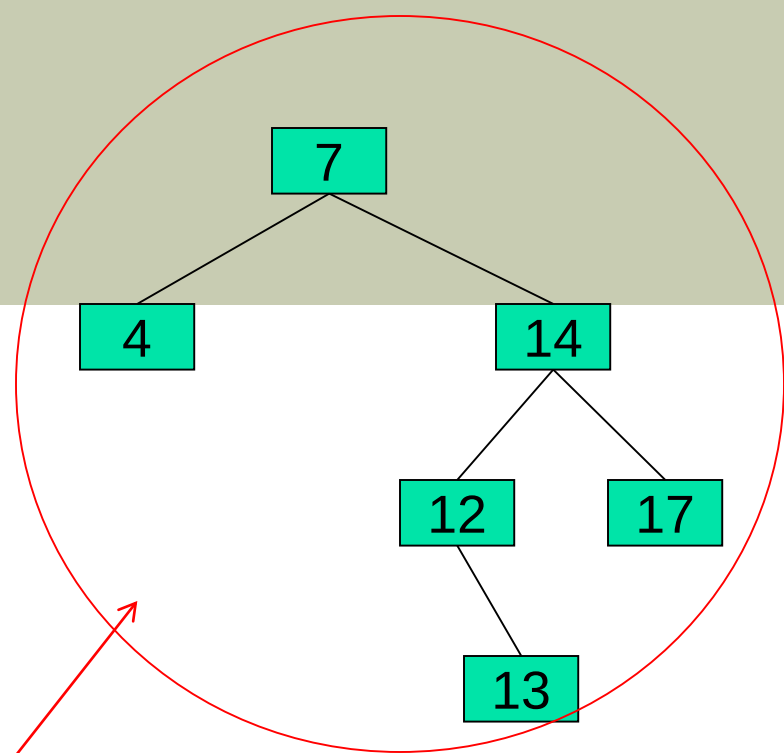


Replace it with the largest
in its left branch

Remove 8



Replace it with
the largest
in its left branch



In Class Exercises

- Build an AVL tree with the following values:
15, 20, 24, 10, 13, 7, 30, 36, 25

Pros and Cons of AVL Trees

Arguments for AVL trees:

1. Search is $O(\log N)$ since AVL trees are **always balanced**.
2. Insertion and deletions are also $O(\log n)$
3. The height balancing adds no more than a constant factor to the speed of insertion.

Arguments against using AVL trees:

1. Difficult to program & debug; more space for balance factor.
2. Asymptotically faster but rebalancing costs time.
3. Most large searches are done in database systems on disk and use other structures (e.g. B-trees).
4. May be OK to have $O(N)$ for a single operation if total run time for many consecutive operations is fast (e.g. Splay trees).

Algorithm Analysis And Data Structures

Questions and Answers

Thank you ...

Prof. Dr. Ayla ŞAYLI

[http://avesis.yildiz.edu.tr/sayli/
sayli@yildiz.edu.tr](http://avesis.yildiz.edu.tr/sayli/sayli@yildiz.edu.tr)